KH정보교육원

애플리케이션 배포

CONTENTS

Part 1. 애플리케이션 배포 환경 구성하기

Charpter 1. 애플리케이션배포환경구성 4

Part 2. 애플리케이션 소스 검증하기

Chapter 2. 애플리케이션 소스검증 및 디버깅 14

Part 3. 애플리케이션 빌드하기

Chapter 3. 애플리케이션 컴파일 & 빌드 27

Part 4. 애플리케이션 배포하기

Chapter 4. 애플리케이션 패키징과 배포 30

Part 1. 애플리케이션 배포 환경 구성하기

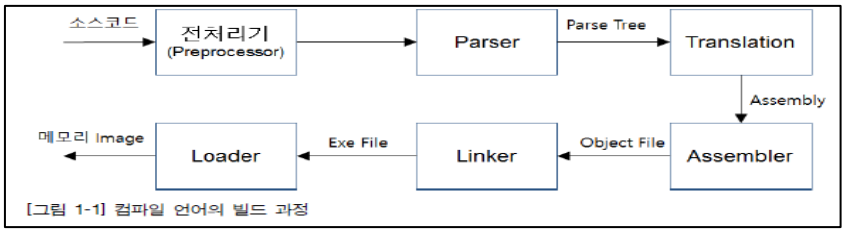
Chapter 1. 애플리케이션 배포 환경 구성

1 소스코드 빌드 과정의 이해

빌드는 소스코드를 실행할 수 있는 상태로 변환하는 과정을 말하며, 프로그래밍 언어의 유형에 따라 빌드 과정이 상이하다.

1) 컴파일 언어(C, C++ 등)

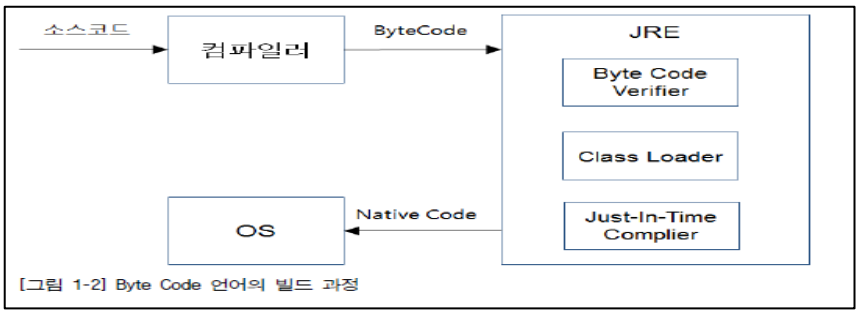
컴파일 언어는 기계어로 바로 변환되어 실행되기 때문에 가장 속도가 빠르고 보안에 유리하다. 하지만 소스 변경 시마다 컴파일 과정을 통해서 빌드 작업을 수행하기 때문에 빌드과정이 오래 걸린다. 빌드 과정은 전처리기, 파싱, 번역, 어셈블리, 링킹 과정을 통해서 진행된다.



[그림 1-1] 컴파일 언어의 빌드 과정

2) Byte Code 언어(Java, C# 등)

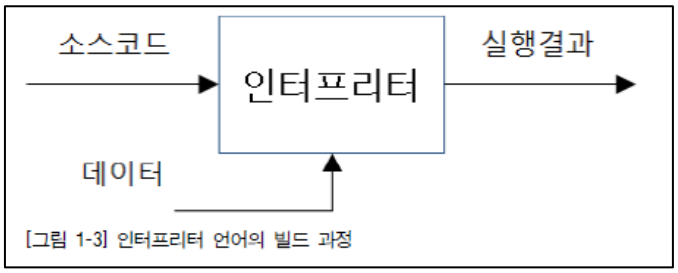
Byte Code 언어는 컴파일의 결과물이 실행파일이 아닌 'class'라는 바이트 코드 파일로생성되고 가상 실행환경인 JRE(Java Runtime Environment), CLI(Common Language Infrastructure)에서 한 줄씩 실행하는 방식으로 빌드된다. JRE, CLI 환경에서 실행될 때로 변환되며, 컴파일 언어에 비해 빌드 과정이 빠르다.



[그림 1-2] Byte Code 언어의 빌드 과정

3) 인터프리터 언어(Javascript, Python, Ruby 등)

인터프리터 언어는 컴파일 언어와 다르게 한 줄씩 번역되어 실행된다. 인터프리터 언어는컴파일하는 과정에서 메모리가 훨씬 적게 소모되고 빠른 시간에 컴파일을 진행할 수 있다.



[그림 1-3] 인터프리터 언어의 빌드 과정

2 애플리케이션 배포(Release) 환경

애플리케이션 배포는 개발자 또는 사용자가 애플리케이션을 실행, 테스트할 수 있도록 컴파일된 프로그램, 실행에 필요한 리소스(이미지, 환경 설정 파일 등)를 서버상의 적합한 위치로 이동하는 작업을 말한다. 그러므로 애플리케이션 배포는 해당 애플리케이션의 동작환경의 영향을 받는다. 웹 애플리케이션의 경우 웹 서버, WAS(Web Application Server)를동작 환경으로 사용한다.

- **웹 서버(Web Server)**

  사용자의 http 요청을 받아 웹 컨테이터에 요청을 전달하고 결괏값을 받아와 사용자에게 전송하는 역할을 수행한다. 애플리케이션 배포 시 이미지, 단순 HTML과 같은 정적인 리소스를 배포하며, 정적 리소스를 빠르고 안정적으로 처리한다.

- **WAS(Web Application Server)**

  사용자의 요청을 받아 동적인 처리를 수행하는 프로그램 실행 부분을 배포한다. WAS 구성 및 운용을 위한 국제 표준 규격을 정하고 있으며, 제품마다 배포 방식과 설정이 일부다르다. 웹 애플리케이션의 경우 UI 배포 영역(JSP, Servlet 등)과 Biz. 배포 영역(EJB, POJO 서비스 등)으로 구분되어 있다.

2.1 웹 서버 환경 구축

가. 리눅스(CentOs)에 Tomcat 설치

- JDK : https://www.oracle.com/technetwork/java/javase/downloads/index.html
  필요한 버전을 다운로드
- Tomcat :
  https://www.oracle.com/technetwork/java/javase/downloads/index.html
  필요한 버전을 다운로드

1. 압축해제

- JDK : "bin"파일을 실행히가 전에 실행 권한 부여

```
[root@test238 ~]# mv jdk-6u41-linux-i586.bin /usr/local
[root@test238 ~]# cd /usr/local
[root@test238 local]# chmod 777 jdk-6u41-linux-i586.bin
[root@test238 local]# ./jdk-6u41-linux-i586.bin
```

- "tar" 명령의 "-C" 옵션으로 압축이 해제될 경로를 지정할 수 있다.

```
[root@test238 ~]# tar zxvf apache-tomcat-6.0.35.tar.gz -C /usr/local
```

- "/etc/profile"을 열어 다음과 같이 입력한다.

```
export JAVA_HOME=/usr/local/jdk1.6.0_41
export PATH=$PATH:$JAVA_HOME/bin
export CLASSPATH=.

export CATALINA_HOME=/usr/local/apache-tomcat-6.0.35
export PATH=$PATH:$CATALINA_HOME/bin
```

- 환경 변수를 완벽하게 등록하기 위해서는 'profile' 다시 읽어줘야 한다.

* 'profile' 파일은 로그인 시 읽혀지는 파일이지만, 재 로그인 과정없이 해줄 수 있다.



- 웹 브라우저에 접속하여 해당 페이지가 뜨면 설치가 완료

2.2 WAS(Web Application Server) 구축

가. CentOs에 Apache와 Tomcat 연동

Tomcat도 단독으로 서비스를 할 수 있으나 보편적으로 Apache와 연동하여 사용하는 경우가 많습니다. 그 이유는 아래와 같습니다.
- 정적컨텐츠의 서비스 속도(이미지 동영상의 경우 Apache가 빠름)
- 여러 대의 Apache와 Tomcat서버의 클러스터링 구성
- ache 웹서버의 다양한 모듈 확장

보안 강화
연동 방식은 mod_jk, mod_proxy, mod_prox_ajp 세가지 방식이 존재
아래는 mod_jk로 진행하겠습니다.

1. mod_jk설치

mod_jk를 설치 하려면 gcc, gcc-c++, httpd-devel 세가지 패키지가 설치되어 있어야 합니다.
아래 명령어를 입력하여 3가지 패키지를 모두 설치

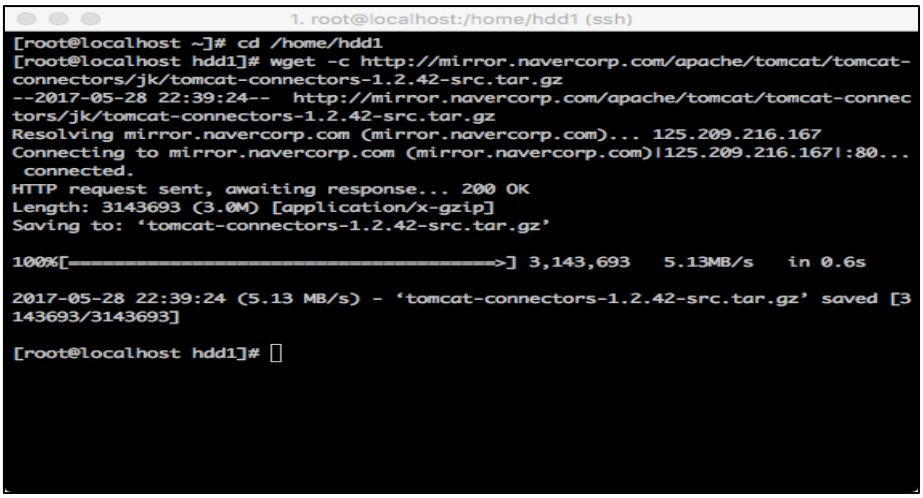
```
$ yum install gcc gcc-c++ httpd-devel
```

1-1. http://tomcat.apache.org/download-connectors.cgi 링크 위치 주소로 방문하여 최신 다운로드



1-2.

```
$ wget -c 링크카피주소
```



다운로드 받은 파일의 확장자가 tar.gz입니다. 이는 압축파일이며, 압축파일을 해제하기 위해 아래의 명령어를 실행

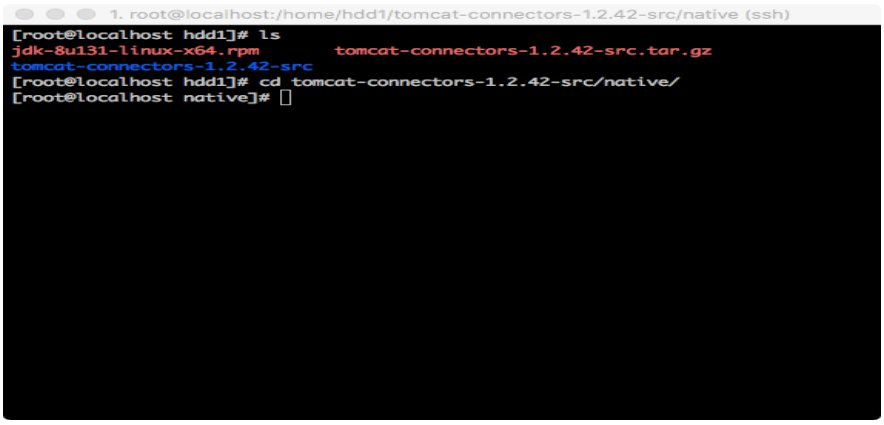
1-3.

```
$ tar zxvf tomcat-connector*
```

압축이 풀리며 디렉토리가 생성된 것을 확인할 수 있습니다.

1-4.

생성된 디렉토리 안에 native디렉토리로 들어갑니다.

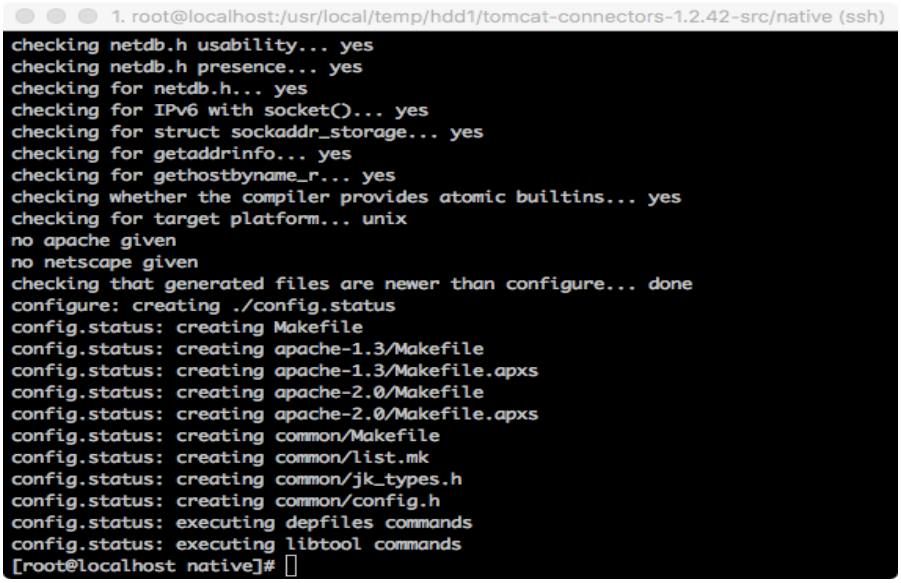


- Makefile을 생성하기 위해 아래 명령어를 실행합니다.
- 옵션 내용은 Apache확장기능 설치를 도와주는 유틸리티의 경로입니다.

- 경로에 찾을수 없다는 에러가 발생하신다면 /usr/sbin의 경로도 확인합니다.

1-5.

```
$ ./configure --with-apxs=/usr/bin/apxs
```



1-6.

- make 명령어로 컴파일을 실행합니다.

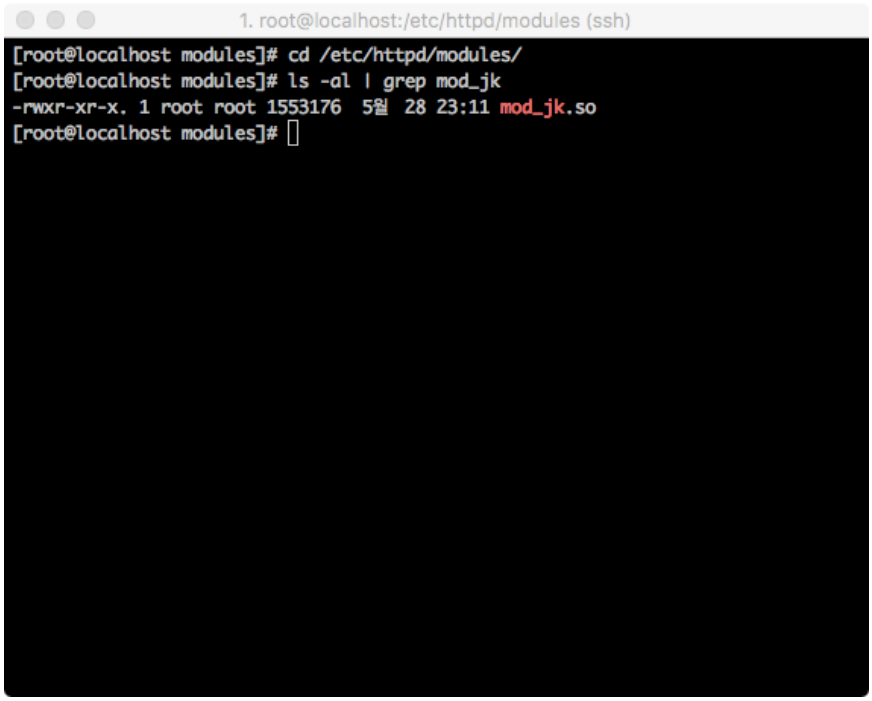
```
$ make
```

- 컴파일 완료후 install 합니다.

1-7.

```
$ make install
```

- install 후 /etc/httpd/modules/ 경로의 파일 안에 mod_jk.so파일이 생성되어있음을 확인할 수 있습니다



1-8.

[Selinux의 보안관련 설정을 변경하기위해, 아래의 명령어를 실행]

```
$ chcon -u system_u -r object_r -t httpd_modules_t /etc/httpd/modules/mod_jk.so
```

2. Apache 설정

2-1

- Apache의 설정은 /etc/httpd/conf/httpd.conf 파일인데 vi 에디터로 오픈합니다.
- vi에디터 창에서 / 를 누르고 LoadModule을 찾아(엔터후 n버튼을 누르면 다음찾기가 됩니다.) 그 아래쪽에 다음의 내용을 추가합니다.

```
LoadModule jk_module modules/mod_jk.so
<VirtualHost *:80>
    ServerName localhost
# 확장자 jsp, json, xml, do를 가진 경로는
```

```
    # woker tomcat으로 연결하는 구문입니다.
        JkMount /*.jsp tomcat
        JkMount /*.json tomcat
        JkMount /*.xml tomcat
        JkMount /*.do tomcat
      </VirtualHost>
```

2-2.

conf.modules.d 경로에 include가 된 것을 확인합니다.
/etc/httpd/conf.modules.d 경로에 진입하여 vi mod_jk.conf 를 실행하여 새로운 설정파일을 오픈

```
<IfModule mod_jk.c>
    # 워커 설정파일 위치
    JkWorkersFile conf/workers.properties
    # 공유 메모리파일 위치 반드시 Selinux 보안때문에 run에 위치 필수
    JkShmFile run/mod_jk.shm
    # log 위치
    JkLogFile logs/mod_jk.log
    # 로그레벨 설정
    JkLogLevel info
    # 로그 포맷에 사용할 시간 형식을 지정한다.
    JkLogStampFormat "[%y %m %d %H:%M:%S] "
 </IfModule>
```

:wq로 저장하고 빠져 나옵니다.

2-3. mod_jk.conf 파일에서 설정한 워커 설정 파일을 생성

설정한 경로: /etc/httpd/conf/workers_jk.properties
vi 로 해당경로 를 오픈하여 파일을 생성

```
$ vi /etc/httpd/conf/workers.properties
```

```
worker.list=tomcat
worker.tomcat.port=8009
worker.tomcat.host=localhost
worker.tomcat.type=ajp13
worker.tomcat.lbfactor=1
```

2-4. Port를 8009로 설정함은 tomcat의 server.xml 파일에서 설정

```
$ vi /usr/share/tomcat/conf/server.xml
```

/etc/httpd/conf/httpd.conf파일을 열어 Apache와 Tomcat Document위치를 맞추어 줍니다.
(맞추어 주지 않으면 jsp 파일은 보이지만, 이미지와 CSS가 로드 되지 않습니다.)
httpd.conf파일을 오픈하여 DocumentRoot를 검색하여 Tomcat의 문서 경로로 변경합니다.
/usr/share/tomcat/webapps/ROOT 입니다

2-5. Directory 권한을 추가합니다.

```
<Directory "/usr/share/tomcat/webapps/ROOT">
    AllowOverride none
    Require all granted
</Directory>
```

2-6. Selinux설정

```
$ chcon -R --type=httpd_sys_rw_content_t /usr/share/tomcat/webapps/ROOT
```

Part 2. 애플리케이션 소스 검증하기

Chapter 2. 애플리케이션 소스 검증 및 디버깅

1 소스코드 검증도구

1.1 검증도구의 구분

소스코드 검증도구는 구현된 SW를 실행하지 않고 테스트하는 정적 테스트 도구와 구현된 SW를 실행하여 동작을 보면서 테스트하는 동적 테스트 도구로 구분한다.



<소스코드 검증도구 구분>

1.2 소스코드 검증도구의 용도

1) 정적 테스트 도구 사용 목적

정적 테스트 도구는 테스트하기 전에 코딩 오류, 성능 저하, 보안 취약점 등의 결함을 조 기에 발견할 수 있도록 지원한다. 이렇게 함으로써 개발의 생산성을 향상시키고, 운영환 경에서 프로그램의 품질 향상을 제고하며, 정량적인 품질 관리시스템을 구축하게 한다.

2) 동적 테스트 도구 사용 목적

테스트 미수행 코드를 확인하고 분기(결정)문 등 특정 유형의 코드 구조가 충분히 테스트 되었는지를 확인하여 추가적인 테스트를 진행함으로써 애플리케이션의 안정성을 제고하고, 소스 품질 관리(통제) 활동을 할 수 있는 정량적인 품질 관리시스템을 구축할 수 있게 한다.

2 코드 인스펙션(Code Inspection)

코드 인스펙션은 정적 테스트의 가장 일반적인 유형으로, 사전에 정의된 코드 작성 규칙(Rule) 기반으로 소스코드를 점검하여 작성 규칙에 위반되는 소스코드를 추출하는 분석 도구로 애플리케이션 개발 시 대부분 사용되며, 빌드 도구와 연계하여 빌드·배포 수행 시 자동적으로 점검할 수 있다.

2.1 코드 인스펙션 Rule 유형

1) 성능 개선

애플리케이션의 성능에 영향을 미칠 수 있는 코드를 점검하는 Rule이다. 메모리 누수, 미사용 변수, 메소드 여부 등을 확인하여 메모리를 낭비하는 코드를 식별한다.

2) 코드 작성 규칙

개발언어에서 사전에 정의된 작성 규칙 또는 프로젝트 내에서 정의된 프로그램 명명 규칙의 준수 여부를 점검하는 Rule이다. 작성 규칙을 미준수한 코드 내역을

추출하여 소스코드의 가독성을 향상시킨다.

3) 에러 발생 가능성

애플리케이션 동작 중 에러 발생 가능성이 있는 코드를 점검하는 Rule이다.

2.2 코드 작성 Rule 심각도 구분(예시)

1) 필수, Blocker

에러 발생 가능성이 매우 높거나 메모리 누수가 발생되는 코드로, 반드시 수정되어야 하는 위반 사항을 말한다.

2) 권고 상, Critical

에러 발생 가능성이 높거나 일반적으로 수정되어야 하는 심각한 위반 사항을 말한다.

3) 권고 중, Major

에러 발생이 있거나 수정을 권고하는 중요 위반 사항을 말한다

4) 권고 하, Minor

소스코드의 가독성, 유지·보수성 향상을 위해 수정을 권고하는 위반 사항을 말한다.

5) 정보, Info

정보성으로 제공되는 위반 사항으로 개발자가 참고하여 적용할 수 있다.

2.3 정규 표현식(Regular Expression)

정규 표현식은 특정한 규칙을 가진 문자열의 집합을 표현하는 범용적인 방식을

말한다. 코드 인스펙션 도구의 코드 작성 규칙은 일반적으로 정규식으로 표현되며, 정규식의 내용 을 수정해서 점검 Rule의 내용을 수정할 수 있다.

메타 문자	의미	설명
.	문자	1개의 문자와 일치한다. 단일행 모드에서는 새줄 문자를 제외한다.
[]	문자 클래스	"["과 "]" 사이의 문자 중 하나를 선택한다. "¦" 를 여러 개 쓴 것과 같은 의미이다. 예를 들면 [abc]d는 ad, bd, cd를 뜻한다. 또한 "-" 기호와 함께 쓰면 범위를 지정할 수 있다. "[a-z]"는 a부터 z까지 중 하나를 의미하고, "[1-9]"는 1부터 9까지 중의 하나를 의미한다.
[^]	부정	클래스 안의 문자를 제외한 나머지를 선택한다. 예를 들면 [^abc]d는 ad, bd, cd는 포함하지 않고 ed, fd 등을 포함한다. [^a-z]는 알파벳 소문자로 시작하지 않는 모든 문자를 의미한다.
^	처음	문자열이나 행의 처음을 의미한다.
$	끝	문자열이나 행의 끝을 의미한다.
()	하위식	여러 식을 하나로 묶을 수 있다. "abc¦dc" 와 "a(b¦)c" 는 같은 의미를 가진다.
\n	일치하는 n 번째 패턴	일치하는 패턴들 중 n 번째를 선택하며, 여기에서 n은 1에서 9 중 하나가 올 수 있다.
*	0회 이상	0개 이상의 문자를 포함한다. "a*b"는 "b", "ab", "aab", "aaab"를 포함한다.
{m, n}	m회 이상 n회 이하	{1,3}b" 는 "ab", "aab", "aaab"를 포함하지만, "b"나 "aaaab" 는 포함하지 않는다.

<정규 표현식 기본 문법(예시)>

3 테스트 프레임워크(동적 분석 도구)의 구성

테스트 프레임워크는 테스트 케이스를 별도의 테스트 코드로 작성하고 동작시킬 수 있는 환경을 제공하는 도구로, 개발자의 반복적이고 시간이 많이 소요되는 테스트 작업을 자동 화하여 테스트에 소요되는 시간과 노력을 절감할 수 있게 한다.

3.1 테스트 프레임워크의 구성

1) 테스트 코드

테스트 코드 작성 및 자동화된 운영환경을 구성한다. 빌드 도구와 연계하여 빌드 수행 시 테스트 코드를 동작시켜 자동화된 테스트 환경을 제공한다

2) 테스트 저장소

테스트 수행을 위한 테스트 코드, 테스트 데이터, 관련 테스트 스크립트, 테스트 수행 결과를 저장, 관리한다.

3.2 Junit 테스트 프레임워크

Java, 오픈소스 기반의 테스트 프레임워크로 Java 개발환경의 범용적인 표준으로 사용되 며(Eclipse 개발 도구에 기본 내장 기능), 다음과 같이 테스트 가능한 Assert 함수를 제공 한다.

제공 함수	설 명
assertArrayEquals(a,b)	배열 a와 b가 일치함을 확인
assertEquals(a,b)	객체 a와 b가 일치함을 확인
assertSame(a,b)	객체 a와 b가 같은 객체임을 확인
assertTrue(a)	a값이 참인지 확인
assertNotNull(a)	객체 a가 null이 아님을 확인

<Junit Assert 구문>

4 수행 순서

1. 코드 인스펙션 점검 Rule을 정의한다.

1) 선정된 코드 인스펙션 점검 도구에서 제공되는 점검 Rule을 확인한다.

코드 인스펙션 점검 도구는 제품에 내장되어 있는 점검 Rule을 갖고 있다. 기본 제공되는 점검 Rule의 내용, 가이드 사항, 심각도, Rule 유형 등을 확인한다.

유형	심각도	Rule명	오류 코드 예시
Bug	Blocker	“.equals()“ should not be used to test the values of “Atomic“ classes	if (aInt1.equals(aInt2)) { ... }
Code Smell	Major	“@Override“ annotation should be used on any method overriding	public boolean doSomething(){...}

Code Smell	Major	“catch“ clauses should do more than rethrow	try {
			s = File.ReadAllText(fileName);
			}
			catch (Exception e) {
			// Noncompliant
			throw e;
			}

Bug	Blocker	Loops should not be infinite	for (;;) { // Noncompliant; end condition omitted
			// ...
			}

<Java 점검 Rule (예시)>

2) 애플리케이션에 정의된 개발 표준을 바탕으로 추가·보완할 점검 Rule이 없는지 확인한다.

일반적으로 코딩 점검 도구에서 기본 제공하는 점검 Rule을 동일하게 반영하지만, 대형 프로젝트 또는 개발 도구, 프레임워크에 의해서 특정 함수를 필수로 사용해야 하 는 경우 또는 애플리케이션 내의 명명 규칙 준수 여부를 체크하기 위해서 점검 Rule을 새롭게 추가, 변경한다.

Rule명	심각도	Rule 점검 요건
주민 번호 하드 코딩 위반	필수	소스코드 내에 주민 번호 식별값을 갖는 내용이 하드 코딩되어 있으면 위반 표시
배치 프로그램 내 특정 함수 사용 금지	필수	Glpost(), LnkOutput(), LogOutput(), Message_Set() 함수 사용 금지 처리
Update 시 특정 칼럼 작성 누락	중요	Update 처리 시 시스템 공통 칼럼 updateDate, updateUser 칼럼이 반드시 포함되도록 처리
공통 프로그램 내 특정 함수 사용 금지	필수	CMTMSET(), TRCMSET(), ERRMSET()

<Java 점검 Rule 추가 요건 사례>

3) 추가·보완이 필요한 점검 Rule을 추가·수정하여 반영한다.

점검 Rule 검토 결과, 추가·수정이 필요한 Rule에 대해 변경 사항을 반영한다. 변경 Rule 이 정규식 변경으로 처리가 가능한 경우에는 해당 정규식 내용을 수정한다. 반영하려는 점검 Rule의 내용이 복잡한 경우에는 별도 개발을 진행한다.

4) 코드 인스펙션 점검 수행을 위한 수행 절차를 정의한다.

- **코드 인스펙션 점검 방식을 정의한다.**

코드 인스펙션의 경우 개발자가 개발 도구상에서 특정 소스만 점검할 수 있고, 애플리 케이션 배포 단계에서 사전 점검으로 점검할 수 있다. 마지막으로, 코드 인스펙션 담당자가 전체 소스를 대상으로 사후 일괄 점검을 수행할 수도 있다. 각각의 점검 방 식 중 해당 애플리케이션 개발 시 적용할 코드 인스펙션 점검

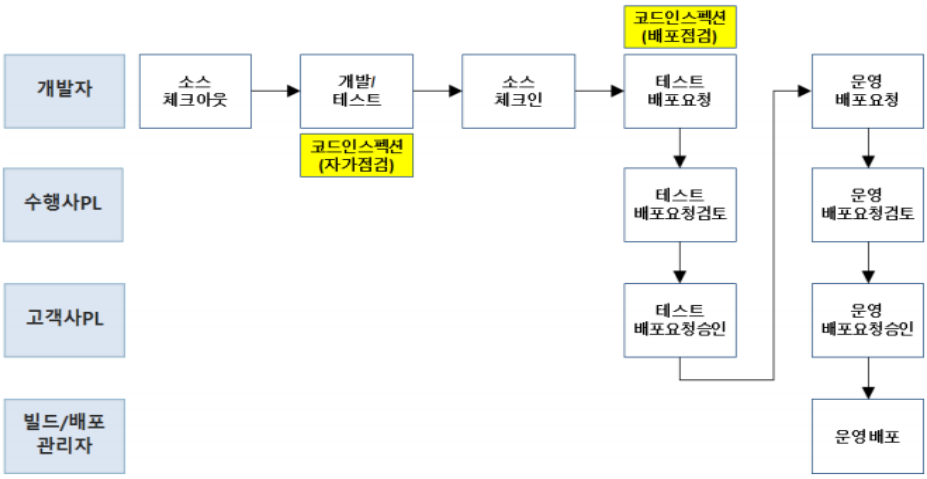
방식을 선정한다.

- **코드 인스펙션 점검 주기를 정의한다.**

코드 인스펙션 담당자가 전체 소스에 대해서 사후 일괄 점검을 수행하는 경우 일괄 점검 주기를 정의한다. 사후 일괄 점검은 구현 상황에 따라서 달라질 수 있는데, 개발 자의 개발 습관을 잡고 코딩 규칙 준수를 위해서 개발 초기부터 적용하는 것이 중요 하고, 2주 ~ 1개월 주기로 주기적인 사후 일괄 점검이 필요하다.

- **코드 인스펙션 점검 수행 절차를 정의한다.**

사전에 정의된 애플리케이션 배포 절차에 코드 인스펙션 점검 단계를 추가하는 방식으로 코드 인스펙션 점검 절차를 정의한다. 코드 인스펙션 점검의 경우 애플리케이션 이 서버에 배포되기 전에 점검되어 오류 사항이 보완될 수 있도록 한다.



<애플리케이션 배포 절차 (예시) (코드 인스펙션 점검 단계 추가)>

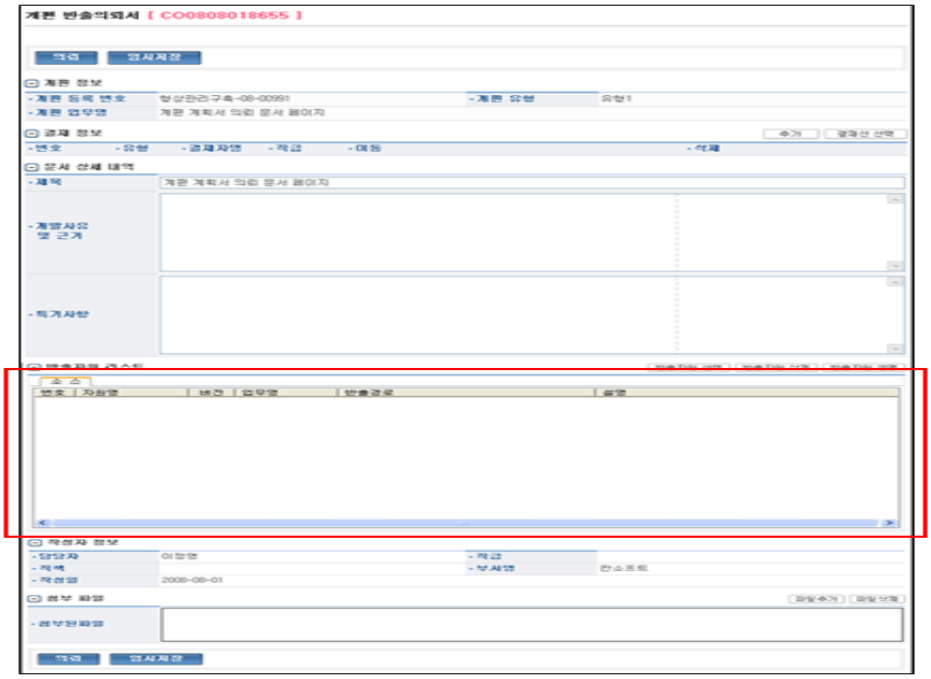
2. 변경 대상이 되는 애플리케이션 소스를 형상관리 서버로부터 반출 (Check Out) 한다.

1) 변경이 요구되는 대상 응용 프로그램, 개편 일정 등을 종합적으로 정리한

개편 계획서를 작성하여 관리자의 승인을 받는다. 이때 개편 유형에 따라 설계 계획서, 설계 검토 의뢰서, 구축 계획서, 구축 검토 의뢰서 등을 선별적으로 함께 작성한다.

2) 변경이 요구되는 (응용 프로그램 소스 등)을 반출하기 위한 반출 의뢰서를 작성하여 관리자의 승인을 받은 후 반출한다.

변경 영향 분석 시 식별된 형상관리 항목에 대해 형상관리자에게 반출 요청을 하여 수정할 자원의 최종 버전을 반출한다.



<반출 의뢰서 (예시)>

3. 코드 인스펙션을 수행하고 점검 결과를 검증한다.

1) 배포 대상인 애플리케이션에 대해서 코드 인스펙션을 수행한다.

코드 인스펙션 도구를 적용하기 전에 점검 Rule에 의한 점검 결과가 정확한지 확인해야 한다. 코드 인스펙션 담당자는 개발된 전체 소스를 대상으로 코드 인스펙션 점검을 수행하고, 코딩 Rule 위반 사항을 추출하여 개발팀에 확인을 요청한다.

Rule명	소스명	위반 내역
포인터 변수의 sizeof 금지	/EFB/sro/EFB10000.c	BSFOPen (lt_BSFCOUNT, ...
빈 함수 점검	/ECL/sco/ECL10001.c	{}
SELECT * 사용 금지	/DDC/DBIO/DDC1400.sql	SELECT * ...
미사용 로컬 함수	/FIM/sro/FIM50003.c	long lt_Term ...

<코드 인스펙션 위반 내역 (예시) >

2) 코드 인스펙션 결과를 확인하고 오탐(False Positive) 여부를 확인한다.

- **코드 인스펙션 결과를 확인한다.**

개발자는 코드 인스펙션 결과를 보고 위반 내역이 정확한지 확인한다. 오탐이 있거나 예외 처리가 필요한 경우 수정을 요청한다

- **개발자의 예외 요청 사항, 검증 내역을 피드백받는다.**

코드 인스펙션 담당자는 개발팀의 오탐 내역, 예외 요청을 접수하여 오탐 내역이 맞는지 확인한다. 예외 요청이 필요한 사항에 대한 패턴을 파악하고 예외 사항을 코드 인스펙션 도구에 반영한다.

Rule명	예외 요청 사항	대상 소스
포인터 변수의 sizeof 금지	프레임워크 자동 생성 프로그램에 대해 대상 제외 필요	프로그램 경로 중 lib 폴더 내 프로그램
하드 코딩 문구(URL)	고객 발송용 이메일 프로그램으로 URL 사용 필요	고객 이메일 프로그램
INSERT문 내 대상 칼럼을 명시하지 않음.	초기 데이터 이행용 프로그램에 대해서 적용 제외 필요	데이터 이행 프로그램

<코드 인스펙션 예외 요청 사항 처리 (예시)>

3) 전체 적용 대상 점검 Rule을 대상으로 심각도, 적용 유무 등 관리 기준을 정의한다.

적용 대상 점검 Rule과 Rule별 위반 내역 건수를 확인하고, 해당 애플리케이션 개발 시 적용할 Rule과 반드시 수정해야 하는 심각도 수준을 정의한다. 정보성, 권고 이하의 심각도의 경우 코딩 규칙 위반 정도가 낮으므로 필수 조치 대상에서 제외할 수 있다.

4) 오탐이거나 보완이 필요한 Rule에 대해 수정한다.

코드 인스펙션 담당자는 검토 결과 오탐 또는 보완이 필요한 것으로 식별된 점검 Rule에 대해서는 오류 원인을 확인하고, 해당 점검 Rule의 내용을 수정한다. Rule 수정 후 위반 내역 점검 결과가 정확한지 재차 확인 후 반영한다.

4. 코드 인스펙션 점검 결과를 반영한 Rule을 가이드하고 개발 조직이 반영하게 한다.

1) 개발팀을 대상으로 코드 인스펙션 점검 결과를 반영한 Rule을 가이드한다.

코드 인스펙션 점검 Rule이 확정되면, 코드 인스펙션 담당자는 점검 Rule을 정리하여 개발팀에 가이드한다. Rule별 상세 설명 및 정상 사례, 오류 사례를 실제 코드를 통해서 제시하여 개발자가 해당 내용을 보고 코드 수정이 가능하도록 제시한다.

Rule명	정상 사례	오류 사례	Rule 상세 설명

빈 함수 점검	if (i > 0) {
	totalCnt = xCount;
	}
	else {
	return false;
	}
		if (i > 0) {
		totalCnt = xCount;
		}
		else {
		//
		}
			if/else 구문 등에 빈 블록이 작성되면 에러 발생 시 찾을 수 없는 로직상의 오류가 발생됨.

switch 문에서 Default 사용 권장	switch(i) {
	case 1:
	dosomething();
	break;
	default
	...
	break;
	}
		switch(i) {
		case 1:
		dosomething();
		break;
		}
			switch 문에는 항상 default 문을 추가해야 함.
			default 문이 없는 경우, 예외 사항 발생 시 예기치 않은 결과 발생 가능함.

INSERT 시 시스템 공통 칼럼 누락	INSERT ... sysdate userid	INSERT ...	시스템 공통 칼럼 누락 시 테이블의 등록, 변경 시간 추적이 불가함.

<코드 인스펙션 점검 Rule 가이드 (예시)>

2) 코드 인스펙션 점검 결과를 파악하고 개발 조직에 전달한다.

코드 인스펙션 점검 Rule이 보완, 확정된 후 코드 인스펙션 담당자는 정해진 주기, 기준에 맞추어서 코드 인스펙션을 수행하고, 그 점검 결과를 개발팀에 전달한다. 코드 인스펙션 도구에 따라서 별도의 웹 페이지를 통해서 코드 인스펙션 점검 결과를 보여 줄 수 있다.

3) 개발 조직의 주요 위반 사항을 정리하여 준수 사항 가이드를 작성, 전달한다.

코드 인스펙션 담당자는 점검 결과 위반 빈도가 높은 주요 코딩 작성 규칙에 대해서 정리하여 개발 조직에게 전달하고 가이드 한다.

5. 도입된 테스트 프레임워크를 기반으로 테스트 코드를 작성하고 테스트를 수행한다.

1) 테스트 코드 작성 기준을 정의한다.

애플리케이션 개발 시 적용한 테스트 코드 작성 기준을 프로젝트의 기간, 진행 상황, 프로그램 중요도, 테스트 유형 등을 고려하여 정의한다. 반복적으로 테스트 가능한 부분과 아닌 부분을 구분하여 기준을 명확히 하고, 구현 단계를 수행하기 전에 개발자에게 가이드한다.

2) 테스트 코드를 작성한다.

개발자는 테스트하려는 프로그램에 적합한 테스트 코드를 작성한다. 프로그램 내의 테

스트하려는 함수에 대해서, 발생 가능한 케이스에 대해서 테스트 케이스를 작성한다.

Junit 테스트 케이스 작성 예시

```
public class FileReaderTest {
        FileReader _input = null;

        @Before
        public void setUp() throws Exception {
                try {
                        _input = new FileReader("C:/Temp/data.txt");
                } catch (FileNotFoundException e) {
                        throw new RuntimeException ("Cannot open the TXT File");
                }
        }

        @Test
        public void testRead() throws IOException {
                char ch = '&';
                for (int i=0; i<4; i++)
                        ch = (char)_input.read();
                assert('d'==ch);
                assertEquals('d',ch);
        }

        @After
        public void tearDown() throws Exception {
                try {
                        _input.close();
                } catch (IOException e) {
                        throw new RuntimeException ("Error in closing TXT File");
                }
        }
}
```

<테스트 케이스 작성 (예시)>

3) 작성한 테스트 코드를 실행한다.

테스트 프레임워크의 실행 기능을 사용하여 테스트 코드를 실행시키고 정상 동작 여부를 확인한다. 테스트를 통과하지 못한 경우 오류 내역, 오류 라인을 확인하고 소스코드를 수정한다. 개발자는 프로그램 개발, 디버깅 및 리팩토링 과정 중에 계속해서 테스트 코드를 실행하여 프로그램의 기능 완성도를 확인한다.

Part 3. 애플리케이션 빌드하기

Chapter 3. 애플리케이션 컴파일 & 빌드

1 빌드란?

컴파일, 검사, 배포등의 과정들이 포함 된다. 즉 소프트웨어가 하나의 단위로서 작동하는지 확인하는 과정이라 할 수 있다.

1.1 빌드의 과정

- 소스코드를 사용할 수 있는 실행 파일로 변환한다(컴파일)
- 정상적으로 작동하는지 테스트한다.
- 실행 파일을 war 파일로 압축한다.(패키징)

이런 과정들이 빌드(Build)라고 할 수 있다.

1.2 종류

1) jar(java archive jar)

jar 압축은 하나의 애플리케이션 기능이 가능하도록 java 파일 등을 압축하는 것이다. 보통 라이브러리 파일들이 jar이다.

2) war(web archive war)

웹 애플리케이션을 지원하기 위한 압축 방식이다. 웹 애플리케이션을 지원하기 위해 jsp, servlet, gif, html, jar 등을 압축 하고 지원해주고 관련된 모든 파일을 패키지화하여 준다는 말이다.

2 필요 지식

2.1 지속적인 통합(CI: Continuous Integration) 환경

1) 빌드 도구

애플리케이션 개발 과정 중 지속적으로 개발된 프로그램을 통합, 빌드, 배포하여 애플리케이션의 개발 내역을 검증, 테스트할 수 있는 환경을 말한다. 통합 빌드 과정 중 테스트 도구, 소스코드 품질 측정도구 등과 연계할 수 있으며, 자동화된 스케줄 관리를 통해서 지속적이고 반복적인 프로그램 빌드, 테스트를 진행할 수 있다.

2) 테스트 도구

개발된 소스코드를 테스트할 수 있는 테스트 코드를 작성, 동작시킬 수 있는 도구로, 통합 빌드 수행 시 연결할 수 있다.(Junit, DBUnit, StrutsTestCase 등)

3) 소스코드 품질 측정도구(코드 인스펙션)

정해진 소스코드 작성 규칙에 따라 소스코드를 점검하고 규칙 위반 여부를 체크하는 도구로, 통합 빌드 수행 시 연결할 수 있다(PMD, FindBugs 등)

4) 테스트 커버리지 측정도구

소스코드 내 테스트 가능한 경로 중 테스트 도구를 통해서 테스트된 커버리지를 측정하는 도구이다(Clover, JCoverage, ElcEmma 등)

5) 빌드 스케줄 관리 도구

작성된 빌드 스크립트를 정해진 조건, 시간에 기동하고 진행 상태, 수행 결과를 관리하는 도구이다(Anthill, CruiseControl, Hudson등).

2.2 테스트 커버리지

1) 테스트 커버리지의 의미

테스트 커버리지는 전체 프로그램의 범위 대비 테스트 수행 시 해당 테스트 수행을 위해 동작된 프로그램의 범위 비율을 의미한다.

2) 테스트 커버리지 측정 유형

- **라인커버리지(또는 구문 커버리지, Statement Coverage)**

개발 소스의 각 라인이 수행되었는지를 확인하는 측정 지표이다.

- **분기 커버리지(Decision Coverage)**

개발소스의 각 분기문이 수행되었는지를 확인하는 측정 지표이다. 만약 소스 내에 if문에 대한 true/false 조건이 있다면, 두가지 경우가 모두 테스트 되어야 100%로 측정된다.

- **조건 커버리지(Condition Coverage)**

각 분기문 내에 존재하는 조건식이 모두 테스트되었는지를 확인하는 측정 지표이다. 조건식 간의 조합에 대해서는 체크하지 않는다.

Part 4. 애플리케이션 배포하기

Chapter 4. 애플리케이션 패키징과 배포

1 애플리케이션 배포

1.1 운영환경의 특징

기업의 IT 인프라 운영환경은 안정적인 IT 서비스 운영 관리를 위해서 애플리케이션 배포 및 변경 작업에 대한 관리, 통제를 강화하고 있으며, 여러 가지 제약사항을 가지고 있다.

1) 네트워크 관점

운영환경은 일반적으로 인터넷망과 분리되어 직접적인 연결을 허용하지 않고 있으며, 별도의 내부망을 구성하고 방화벽을 토해서 인가된 IP를 통해서만 접근을 허용한다. 기업의 운영환경, 서버에 접근하기 위해서는 사전에 방화벽 허용 신청을 통해서 배포 서버의 IP가 접근 가능하도록 설정되어야 한다.

2) 계정 관리 부문

정부의 보안 관리 기준에 따라 정기적으로 패스워드(password)를 변경하는 형태로 관리된다. 그러므로 애플리케이션 배포 전에 운영 서버에 접근 가능한 계정, 프로토콜, 접근 디렉터리를 정해서 신청해야 하며, 아이디, 패스워드가 소스코드 또는 빌드 스크립트 상에 하드 코딩되지 않도록 주의해야 한다.

3) 보안 취약점 부문

해킹 위협의 증가로 운영환경은 다양한 보안 취약점 점검을 주기적으로 수행하고 보안 취약점을 보완한다. 애플리케이션 운영 배포 시 이러한 보안 취약점은 애풀리케이션 관점에서도 점검해야 한다.

1.2 프로그램 통제 부문 전자 금융 감독 규정

금융권 애플리케이션의 경우 안정적인 애플리케이션 운영 및 변경 작업을 수행하기 위해서 프로그램 통제 부문에 대한 기본 규정을 제시하고 있으며, 금융 애플리케이션을 운영환경에 배포/운영하기 위해서는 해당 기준을 준수하는 애플리케이션 배포/운영환경과 절차를 구현해야 한다.

1) 적용 대상 프로그램 종류 및 등록/변경/폐기 방법을 마련할 것 (운영 절차 수립)
2) 프로그램 변경 전후 내용을 기록/관리 할 것 (변경 내역에 대한 이력 관리)
3) 프로그램 등록/변경/폐기 내용의 정당성에 대해 제 3자의 검증을 받을 것 (소스 변경내역, 테스트 수행 결과에 대한 검증)
4) 변경 필요시 해당 프로그램을 개발 또는 테스트 시스템으로 복사 후 수정할 것 (운영환경에서 직접 변경 금지)
5) 프로그램에 대한 접근은 업무 담당자에 한정할 것 (소스에 대한 개발자별 접근 관리 적용)
6) 운영 시스템 적용은 처리하는 정보의 기밀성, 무결성, 가용성을 고려하여 충분한 테스트 및 관련 책임자 승인 후에 실시할 것 (상위 관리자의 승인 절차, 테스트 결과 확인)
7) 프로그램 반출, 실행 프로그램의 생성 및 운영 시스템 등록은 전산 자료 관리자 등 해당 프로그램 담당자 이외의 사람이 수행할 것 (개발자와 배포 담당자 분리)
8) 운영체제, 데이터베이스 관리 프로그램 등의 시스템 프로그램도 응용프로그램과 동일한 수준으로 관리할 것 (시스템 프로그램, 시스템 SW에 대해서도 동일 수준으로 관리)
9) 프로그램 설명서, 입출력 레코드 설명서, 프로그램 목록 및 사용자/운영자 지침서 등 프로그램 유지보수에 필요한 문서를 작성/관리할 것 (프로그램 변경 시 관련 설계 산출물, 메뉴얼에 대해서도 업데이트 수행)
10) 전자 금융 거래에 사용되는 전산 프로그램은 실제 업무를 처리하는 정보 처리 시스템에 설치하기 전에 자체 보안성 검증을 실시할 것 (인프라, 소스코드 보안 취약점 점검)

1.3 수행 순서

1. 애플리케이션 실행 환경에 대한 정보를 확인한다.

1) 애플리케이션 배포환경을 확인한다.

애플리케이션이 배포된 운영환경의 서버 IP, 디렉토리, 접근 계정을 확인한다. IP 사용 신청 및 접근 계정이 아직 부여되지 않은 경우 인프라 관리자에게 신청한다.

2) 애플리케이션 배포환경의 제약 사항을 확인한다.

운영환경 상의 서버 관리 기준, IP관리 기준, 보안 기준 등을 확인하여 애플리케이션 배포 시 문제가 발생될 수 있는 사항을 확인한다. 파일 직접 쓰기가 필요하거나 파일 전송이 필요한 경우, 인프라 관리자에게 별도의 계정을 사전에 부여받도록 한다.

2. 애플리케이션 배포 절차에 따라 운영 환경에 적용한다.

1) 이행 의뢰서를 작성한다.

이행 의뢰서의 자원 및 스크립트에서 이행할 자원을 추가한다.

2) 이행 작업이 정상적으로 되었는지 작업 결과를 확인한다.

이행 작업이 완료되고 난 후에 Agent 또는 FPT를 사용한 운영 서버로의 배포 결과를 확인하고, 대상 장비에 빌드 스크립트가 존재할 경우 빌드 스크립트 동작 결과를 확인한다.

3. 애플리케이션 배포 후 정상적으로 작동하는지를 확인한다.

1) 배포 담당자가 배포 결과를 확인한다.

배포 담당자는 배포 수행 결과상 오류 사항이 없는지 확인한다. 배포 시 오류가 발생된 경우 해당 로그 정보를 확인하고 문제 사항을 즉시 해당 개발자에게 전달하여 수정을 요청한다.

2) 개발자가 배포 결과를 확인한다.

개발자는 배포 완료 후 반영된 프로그램이 정상적으로 반영되었는지 테스트를 수행한다. 사전에 준비된 테스트 케이스, 체크리스트를 기반으로 테스트를 수행하고, 변경된 부분이 정상적으로 동작하는지를 확인한다.

4. 애플리케이션 배포 결과 문제가 발생했을 경우 적용 내용을 이전 상태로 복원한다.

1) 오류 원인을 확인한다.

개발자는 배포된 프로그램상에 오류가 발생된 경우 오류원인을 파악하여 프로그램 개발상에 문제가 있는 것인지, 배포 시 문제가 발생할 것인지를 확인한다. 필요시 배포 담당자를 포함하여 관련 개발자와 협업을 통해서 오류 원인을 파악한다.

2) 배포 작업을 이전 상태로 복원한다.

오류 원인 분석 후 프로그램 재수정 또는 배포작업을 다시 해야 하는 경우, 기배포된 프로그램을 롤백 처리한다. 롤백 처리가 어려운 경우 해당 프로그램을 수정된 내용을 반영하여 재배포한다.

5. 애플리케이션 운영환경 전환(이관) 방안을 수립하고 애플리케이션 운영 전환을 수행한다.

1) 신규 시스템 운영 환경 전환(이관) 프로세스를 설계한다.

- **운영 및 유지보수 전환 기준을 정의한다.**

신규 프로젝트 종료 후 운영 및 유지보수로 전환을 위한 필수 산출물을 정의한다. 필수 산출물에는 화면 설계서, 데이터 설계서, 프로그램 설계서, 소스코드, 운영자 메뉴얼 등이 포함된다.

- **운영 및 유지보수 전환을 준비한다.**

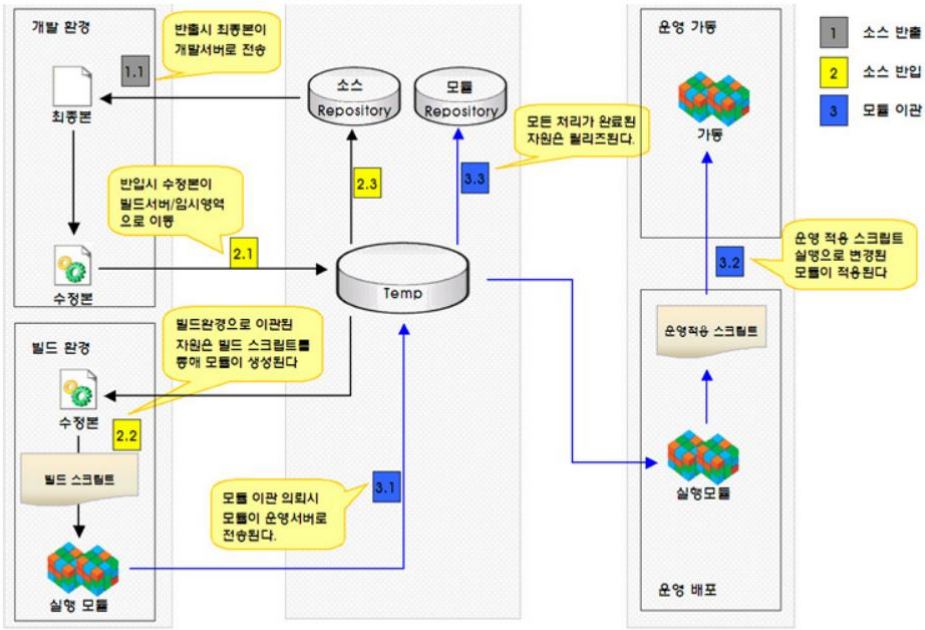
전환 수행 조직별 역할 및 책임, 전환 대상, 전환 일정, 교육 및 인수인계 방안, 운영 주체 변경 시점, 이슈 및 리스크, 체크리스트 등을 포함한 운영 및 유지보수 전환 계획을 수립하고 관리자로부터 승인을 받는다.

2) 운영 및 유지보수 전환을 수행한다.

전환 대상 항목과 실물의 일치 여부를 확인하고, 부적합 사항에 대해서는 인계 담당자(개발자)에 알려 즉시 시정 조치한다. 또한 운영 환경으로 전환되는 시스템에 대한 정보를 유지보수 관리시스템(ITSM 시스템), 형상관리 시스템 등에 반영시킨다.

3) 운영 및 유지보수 전환을 종료한다.

운영 및 유지보수 이관 대상 서비스별로 인수인계서를 작성하여 관련자들이 서명한다. 인수인계서에는 인수인계에 대한 책임, 인수인계 대상 업무/활동 내역, 관련 산출물 목록, 인수인계 담당자 등의 내용이 포함되어야 한다.



[그림 4-1] 운영환경에서의 애플리케이션 빌드·배포환경 (예시)